



可变参数模板

▼ 基本概念

可变参数模板（variadic templates）是 C++ 11 引入的一个强大的特性。它允许模板函数或模板类接受可变数量的参数。

可变参数模板使用 `...` 语法定义：

```
template<typename... Args>
void func(Args... args) {
    // 处理参数包
}
```

C++

▼ 参数包

参数包 `Args...` 本身并不是一个具体的容器，而是一种语法概念，用于表示零个或多个参数。参数包在编译时展开，并不会以某种标准容器（如 `std::tuple` 或 `std::vector`）的形式存储，而是由编译器在需要时进行拆解或转发。

所以参数包的性质与之前所了解的容器并不相同，其不能被直接访问，甚至不能直接作为类的成员。如果想要作为类的成员，需要使用容器封装，如 `std::tuple`（可接受不同类型参数）。

当我们定义一个可变参数模板时，`Args...` 本质上是一个类型列表，它并不会被存储在某个具体的容器里，而是由编译器在实例化模板时展开。

参数包主要分为以下类型：

- 类型参数包（Type Parameter Pack）
- 非类型参数包（Non-Type Parameter Pack）
- 模板模板参数包（Template Template Parameter Pack）

最常使用的 `template<typename... Args>`，属于类型参数包，`Args` 是一个类型列表。

由类型参数包定义的变量则称为**对象参数包**，比如 `Args... args` 中的 `args`。

在定义时参数包时，要加上 `...`，表示你定义的类型或者对象是参数包，`typename... Args` 表示 `Args` 是类型参数包，`Args... args` 表示 `args` 是对象参数包。如果是参数包引用的话，`...` 加在 `&` 之后，`Args&&...`。

同理，`sizeof...` 用来返回参数包中参数的个数。

在使用参数包时可以在参数包后加上 `...` 来表示**包展开**（参数包与 `...` 之间如果存在逗号则是折叠表达式），此时参数包会被展开为一段逗号分隔的序列，该序列不属于任何合法的类型，所以不能参与各种运算或执行，一般只能用来进行参数传递。需要对参数包进行运算，则需要使用折叠表达式。

包展开在某些地方类似于宏展开的字符串替换，但是它比宏更高级的地方在于“针对每个元素重复特定模式”。

除此之外，语法规则规定，包展开 `...` 必须出现在特定的“接收列表”的上下文中（如函数参数、初始化列表）。像 `int x = (args...);` 这种情况，即使 `args...` 展开为一段 `1, 2, 3` 这样的序列后，看起来像是一个合法的逗号运算符表达式，也会被认为**语法歧义或操作数序列位置不当**而报错。

▼ 递归展开

模板参数中使用参数包时，参数包应放在最后，接受不确定数量的参数。

利用这个特点可以通过递归方式展开参数包：

```
C++

void print() {
    std::cout << "结束展开" << std::endl;
}

template<typename T, typename... Args>
void print(T first, Args... args) {
    std::cout << first << " ";
    print(args...);
}
```

不断递归调用函数，传递参数包时，参数包会被自动展开，第一个参数被 `first` 接受，剩下的参数再次被参数包接受，直到参数包为空时中止。

通过这个性质与偏特化，可以获取参数包中某个参数的索引：

```
C++

template<typename T, typename... Ts>
struct index_of;

template<typename T, typename... Ts>
struct index_of<T, T, Ts...> {
    static constexpr std::size_t value = 0;
};

template<typename T, typename U, typename... Ts>
struct index_of<T, U, Ts...> {
```

使用 `index_of<T, Ts...>::value` 可以获得 `T` 类型在参数包 `Ts...` 中的索引。

每次调用 `index_of` 时，参数包的首个参数都会匹配至偏特化模板中的第二个模板参数中，若前两个模板参数不同，则筛掉此时的二个模板参数，索引计数 `+1`，继续递归调用，直至前两个模板参数相同，此时 `value` 即第一个模板参数 `T` 在参数包 `Ts...` 中的索引。

在可变参数模板中，因为不知道具体使用的类型的数量，所以不可能提前声明好每个类的实例作为成员，又因为参数包中可能存在不同的类型，所以一般使用 `std::tuple` 来存储不同类的实例。

此时要从 `std::tuple` 中获得某个类的实例，需要定位该类在参数包中的索引。

```
C++
template<typename... Cs>
struct view {
    std::tuple<storage<Cs>*> pools;

    template<typename T>
    storage<T>* pool() {
        constexpr std::size_t i = index_of<T, Cs...>::value;
        return std::get<i>(pools);
    }
};
```

▼ 可变参数偏特化

有时主模板中没有可变参数，但是在偏特化时可以添加可变参数：

```
C++
template<typename T>
struct MyStruct;

template<typename... Args>
struct MyStruct<std::tuple<Args...> > {
    // 这里的 Args... 就被成功捕获了
};

// 使用时：
MyStruct<std::tuple<int, double>> s;
```

常用于针对 `std::tuple` 进行偏特化的场合，主模板可以接受一个模板参数，但是 `std::tuple` 作为可变参数模板不能匹配 `typename T`，所以使用 `typename... Args` 进行偏特化。

▼ 折叠表达式

C++17 引入了折叠表达式，可以直接展开参数包。

无论是对象参数包（值参数包）还是类型参数包，都可以触发折叠表达式，但对象参数包可以直接用于计算，而类型参数包通常用于元编程。

在折叠表达式遇到参数包时，会对其进行展开。

如果同时存在多个参数包，则会同时展开，取相同索引的参数，所以这要求参数包的参数数量一致。

如果不希望不同参数包同时展开，可以通过创建一个新的可变参数模板，只展开一个参数包，然后与其他不展开的参数包一起传入新的模板。

在参数包后加上 `...`，即表示参数包不展开，而是作为整体传递。

```
C++

template <template <typename> class... Containers, typename T >
void printOne(T x) {
    // 为每个容器类型创建一个对象
    ([&]() {
        Containers<T> container{ x }; // 初始化容器
        std::cout << typeid(container).name() << std::endl;
    }()), ...);
    std::cout << std::endl;
}

template <template <typename> class... Containers, typename... Args>
void printAll(Args&&... args) {
    // 展开参数包并调用 printOne
    ([printOne<Containers...>(args)), ...);
}
```

该例中只展开了 `args` 对象参数包，`Containers...` 类型参数包作为整体传递。

折叠表达式分为一元和二元形式：

- 一元左折叠：`(... op args) → ((arg1 op arg2) op arg3)...`
- 一元右折叠：`(args op ...) → (arg1 op (arg2 op arg3))...`
- 二元左折叠：`(init op ... op args) → (((init op arg1) op arg2)...)...`
- 二元右折叠：`(args op ... op init) → (arg1 op (arg2 op (... op init)))`

通过折叠表达式，可以实现参数包的展开。

需要注意的是，在折叠表达式中，逗号一般不再是分隔符的作用，而是作为一个运算符使用。

逗号运算符的作用是保证求值顺序，比如 `int x = (1, 2, 3);`，这里逗号作为运算符，保证了 `(1, 2, 3)` 的返回结果为最后一位的 `3`，所以 `x=3`。

通过折叠表达式依次输出参数包的方法一般有两种：

- `((std::cout << args << " "), ...);`：一元右折叠语法，使用逗号运算符，其实并不需要返回值，只是确保左侧代码会被执行。代码展开后的形式类似 `((std::cout << args1 << " "), ((std::cout << args2 << " "), ((std::cout << args3 << " "), (...))))`，但其实交换两者顺序后的输出相同，`(... , (std::cout << args << " "));` 展开后只是括号的结构有所不同，但是 `args1`、`args2`、`args3...` 的相对顺序没有变，而逗号运算符保证了先执行左侧命令，所以输出的结果不变。
- `(std::cout << ... << args) << std::endl;`：二元左折叠语法，虽然可以按顺序输出，但是无法在输出之家增加例如空格的其他内容。如果使用 `(std::cout << ... << (args << ' ') << std::endl;` 这种语法，那么其展开类似于 `((std::cout << args1 << ' ') << (args2 << ' ')) << (args3 << ' ') ... << std::endl;` 此时由于括号内的运算优先级更

高，会运算 `(args << ' ')`，因为 `std::string` 没有定义 `operator<<` 来拼接 `const char*`，所以会报错。

总结两种方法，更推荐第一种，逗号运算符保证了从左到右的运算顺序，而不用担心括号运算符影响运算优先级。

▼ 非类型参数包

如果需要单一类型但不确定数量的常量，可以使用非类型参数包：

```
C++
template<int... nums>
void print() {
    ((std::cout << nums << " "), ...);
}

int main() {
    print<1, 2, 3, 4>(); // 输出: 1 2 3 4
    return 0;
}
```

这种用法的特点是，模板参数不再是一个类型列表，而是一个常量列表。不同于 `print<int, int>`，而是将具体常量作为模板参数 `print<1, 2>`。

二者在定义上的区别在于：类型参数包使用 `typename` 或 `class`，而非类型参数包使用具体的类型、`size_t` 或者 `auto`，也可以是同时使用模板定义的参数类型(不能是参数包，而是单个类型)：

```
C++
template<typename T, T... values>
void print() {
    // 打印非类型参数包中的每个元素
    ((std::cout << values << " "), ...);
    std::cout << std::endl;
}

template<typename T, T*... ptrs>
void print_pointers() {
    ((std::cout << ptrs << " "), ...);
    std::cout << std::endl;
}

template<auto... Args>
void printNonTypes() {
    ((std::cout << Args << " "), ...);
    std::cout << std::endl;
}
```

▼ 模板模板参数包

模板模板参数包是模板参数包的一种扩展，也可以叫做二阶模板参数包，它接受其他模板作为参数，而不是普通的类型或者常量值。也就是说，它能够接受一个或多个模板类型作为参数，并且这些模板本身还可以有自己的模板参数。

C++

```
template<template<typename...> class... Templates>
void foo(Templates... templates) {
    // 处理每个模板类型
}
```

该例中 `template<typename...> class` 表示为一阶可变参数模板，之后的 `...` 表示该模板可以是一个参数包，可以接受多个可变参数模板。

如果使用 `template<typename> class`，则表示该参数包只能接受多个含有一个模板参数的模板类

C++

```
template<template<typename> class... Containers>
void printContainers() {
    // 打印每个容器的内容
    ((Containers<int> container{10, 20, 30},
      for (const auto& item : container) {
          std::cout << item << " ";
      },
      std::cout << std::endl, ...));
}

int main() {
    printContainers<std::vector, std::list>(); // 输出: 10 20 30 \n 10 20 30
    return 0;
}
```

该例中 `Containers` 作为类型参数包，被 `...` 展开。如果想遍历类型参数包中的类型，并进行较复杂的操作时，可以使用 `lambda` 表达式与折叠表达式 `lambda, ...`。也可以创建一个新的模板函数，用来接受折叠表达式展开的类型参数包：

C++

```
template<template<typename> class Container>
void addElementsToContainer() {
    Container<int> container;
    container.push_back(1);
    container.push_back(2);
    container.push_back(3);

    std::cout << "Container elements: ";
    for (const auto& item : container) {
        std::cout << item << " ";
    }
    std::cout << std::endl;
}

template<template<typename> class... Containers>
void addToAllContainers() {
    // 使用折叠表达式处理多个容器类型
    (addElementsToContainer<Containers>(), ...);
}
```

```
int main() {
    addToAllContainers<std::vector, std::list>(); // 输出: Container elements: 1 2
    3 \n Container elements: 1 2 3
    return 0;
}
```

模板模板参数包也可以实现递归处理：

```
template<template<typename> class T>
void process() {
    T<int> container = {1, 2, 3, 4};
    for (const auto& item : container) {
        std::cout << item << " ";
    }
    std::cout << std::endl;
}

template<template<typename> class First, template<typename> class... Rest>
void process() {
    process<First>(); // 处理第一个模板

    // 递归处理剩余模板
    process<Rest...>();
}
```

C++